

Question 1 — Inventory Management System

Source code:

```
#include<iostream>

#include <iomanip>

using namespace std;

class Product{
    int productId;

    string name;

    double price;

    int quantity;

public:

    void acceptDetails();

    void displayDetails() const;

    double totalValue()const;

    bool isLowStock(int threshold) const;

};

void Product::acceptDetails(){

    cout<<"Enter Product ID: ";

    cin>>productId;

    cout<<"Enter Name: ";

    cin>>name;

    cout<<"Enter Price: ";

    cin>>price;

    cout<<"Enter the Quantity: ";

    cin>>quantity;

}

double Product::totalValue()const{

    return price*quantity;

}

void Product::displayDetails()const{

    cout<<left<<setw(8)<<productId<<setw(15)<<name<<setw(12)<<fixed<<setprecision(2)

    <<price<<setw(8)<<quantity<<setw(15)<<fixed<<setprecision(2)<<"(Rs.

    "<<totalValue())<<")";
```

```

}

bool Product::isLowStock(int threshold) const{
    return quantity<threshold;
}

double reorderCost(int qty,double unitPrice){
    return qty*unitPrice;
}

double reorderCost(double qty,double unitPrice){
    return qty*unitPrice;
}

double reorderCost(int qty,double unitPrice,double taxrate){
    double cost=qty*unitPrice;
    double tax=cost*taxrate/100;
    return cost+tax;
}

double applyDiscount(double price,double discountPercent=10.0){
    double discount=price*discountPercent/100;
    return price-discount;
}

int main(){
    Product arr[5];
    for(int i=0;i<5;i++){
        arr[i].acceptDetails();
    }

    cout<<"===== INVENTORY REPORT ====="<<endl;
    cout<<left<<setw(8)<<"ID"<<setw(15)<<"Name"<<setw(12)<<"Price"<<setw(8)
        <<"Qty"<<"Total Value"<<endl;

    for(int i=0;i<5;i++){
        arr[i].displayDetails();
        cout<<endl;
    }

    int highestTotalValue=0;
    for(int i=1;i<5;i++){

```

```

        if(arr[i].totalValue()>arr[highestTotalValue].totalValue()){

            highestTotalValue=i;

        }

    }

    cout<<"Highest Value Product: ";

    arr[highestTotalValue].displayDetails();

    cout<<endl;

    int threshold;

    cout<<"Enter threshold: ";

    cin>>threshold;

    for(int i=0;i<5;i++){

        if(arr[i].isLowStock(threshold)){

            arr[i].displayDetails();

            cout<<endl;

        }

    }

    cout << "Reorder Cost (int quantity): "

        << reorderCost(10, 100.0) << endl;

    cout << "Reorder Cost (double quantity): "

        << reorderCost(10.5, 100.0) << endl;

    cout << "Reorder Cost with Tax: "

        << reorderCost(10, 100.0, 18.0) << endl;

    cout << "Discounted Price (default 10%): "

        << applyDiscount(1000.0) << endl;

    cout << "Discounted Price (20%): "

        << applyDiscount(1000.0, 20.0) << endl;

    return 0;

}

```

Output:

```

===== INVENTORY REPORT =====
ID      Name      Price      Qty      Total Value
101     Widget-A      250.00     120     (Rs.      30000.00)
102     Bolt-B        15.50       8       (Rs.      124.00)
103     Screw-C       10.00      50       (Rs.      500.00)
104     Motor-D       500.00     20       (Rs.     10000.00)
105     Nuts-E         8.00       8       (Rs.       64.00)
Highest Value Product: 101      Widget-A      250.00     120     (Rs.      30000.00)
Enter threshold: 10
102     Bolt-B        15.50       8       (Rs.      124.00)
105     Nuts-E         8.00       8       (Rs.       64.00)
Reorder Cost (int quantity): 1000.00
Reorder Cost (double quantity): 1050.00
Reorder Cost with Tax: 1180.00
Discounted Price (default 10%): 900.00
Discounted Price (20%): 800.00

```

Question 2 — Hospital Patient Registry

Source code:

```
#include<iostream>

#include<string>

using namespace std;

class Patient{

    int patientId;

    string name;

    int age;

    string ward;

    const string bloodGroup;

public:

    Patient():patientId(0),name("Unknown"),age(0),ward("General"),bloodGroup("O+"){

        cout<<"[Constructor] Default patient registered." << endl;

    }

    Patient(int id,const string& name):patientId(id),name(name),age(0),ward("Emergency"),bloodGroup("O+"){

        cout<<"[Constructor] Emergency: "<<name<< endl;

    }

    Patient(int id,const string& name,int age,const string& ward,const string& bg):

        patientId(id),name(name),age(age),ward(ward),bloodGroup(bg){

        cout<<"[Constructor] Full admission: "<<name<<endl;

    }

    ~Patient(){

        cout<<"[Destructor] Patient: "<<name<<" discharged."<<endl;

    }

    void displayRecord()const{

        cout<<"Patient Record: "<<endl;

        cout<<"ID : "<<patientId<<endl;

        cout<<"Name : "<<name<<endl;

        cout<<"Age : "<<age<<endl;
```

```

        cout<<"Ward : "<<ward<<endl;

        cout<<"Blood Group : "<<bloodGroup<<endl;
    }

    void transferWard(const string& newWard){
        cout<<"Ward transfer : "<<name<<"->"<<newWard<<endl;

        ward=newWard;
    }
};

int main(){
    Patient p1;

    Patient p2(102,"Sanjay",52,"General","O+");

    Patient p3(101,"Pranav");

    Patient* patients[4];
    for(int i=0;i<4;i++){
        int id,age;

        string name,ward,bg;

        cout<<"Enter Details for the patient: "<<endl;

        cout<<"Enter ID: ";

        cin>>id;

        cout<<"Enter Name: ";

        cin>>name;

        cout<<"Enter Age: ";

        cin>>age;

        cout<<"Enter Ward: ";

        cin>>ward;

        cout<<"Enter Blood Group: ";

        cin>>bg;

        patients[i] = new Patient(id, name, age, ward, bg);
    }

    for (int i = 0; i < 4; i++){
        patients[i]->displayRecord();
    }
}

```

```

    }

    patients[1]->transferWard("ICU");

    for (int i = 0; i < 4; i++){

        delete patients[i];

    }

    return 0;

}

```

Output:

```

[Constructor] Default patient registered.
[Constructor] Full admission: Sanjay
[Constructor] Emergency: Pranav
Enter Details for the patient:

Patient Record:
ID : 101
Name : Pranav
Age : 21
Ward : Genral
Blood Group : B+
Patient Record:
ID : 102
Name : Sanjay
Age : 52
Ward : General
Blood Group : O+
Patient Record:
ID : 103
Name : Manisha
Age : 42
Ward : Emergency
Blood Group : B+
Patient Record:
ID : 104
Name : Tejashree
Age : 24
Ward : Special
Blood Group : A+
Ward transfer : Sanjay->ICU
[Destructor] Patient: Pranav discharged.
[Destructor] Patient: Sanjay discharged.
[Destructor] Patient: Manisha discharged.
[Destructor] Patient: Tejashree discharged.
[Destructor] Patient: Pranav discharged.
[Destructor] Patient: Sanjay discharged.
[Destructor] Patient: Unknown discharged.

```

Question 3 — Game Engine Entity System

Part A — this Pointer and Method Chaining

Source code:

```
#include<iostream>

using namespace std;

class Entity{
    string name;
    int health;
    int level;
    string type;
public:
    int getHealth() const {
        return health;
    }

    Entity& setHealth(int health) {
        this->health = health;
        return *this;
    }

    int getLevel() const {
        return level;
    }

    Entity& setLevel(int level) {
        this->level = level;
        return *this;
    }

    const string& getName() const {
        return name;
    }

    Entity& setName(const string &name) {
        this->name = name;
        return *this;
    }
}
```

```

    const string& getType() const {

        return type;

    }

    Entity& setType(const string &type) {

        this->type = type;

        return *this;

    }

    void displayInfo()const{

        cout<<"Name: "<<name<<endl;

        cout<<"Health: "<<health<<endl;

        cout<<"Level: "<<level<<endl;

        cout<<"Type: "<<type<<endl;

    }

};

int main(){

    Entity player;

    Entity enemy;

    Entity item;

    player.setName("Aragorn").setHealth(100).setLevel(10).setType("Player");

    enemy.setName("Orc").setHealth(60).setLevel(5).setType("Enemy");

    item.setName("HealthPotion").setHealth(0).setLevel(1).setType("Item");

    player.displayInfo();

    cout<<endl;

    enemy.displayInfo();

    cout<<endl;

    item.displayInfo();

    cout<<endl;

    return 0;

}

```

Output:


```
Name: Aragorn
Health: 100
Level: 10
Type: Player

Name: Orc
Health: 60
Level: 5
Type: Enemy

Name: HealthPotion
Health: 0
Level: 1
Type: Item
```

Part B — Namespaces

Source code:

```
#include<iostream>

using namespace std;

namespace Physics{

    double clamp(double val, double min, double max);

    double lerp(double a, double b, double t);

}

namespace GameMath{

    int clamp(int val,int min,int max);

    double lerp(double a,double b,double t);

}

double Physics::clamp(double val,double min,double max){

    if(val<min)

        return min;

    if(val>max)

        return max;

    return val;

}

double Physics::lerp(double a,double b,double t){

    return a+(b-a)*t;

}

int GameMath::clamp(int val,int min,int max){
```

```

        if(val<min)
            return min;

        if(val>max)
            return max;

        return val;
    }

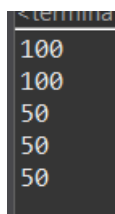
    double GameMath::lerp(double a,double b,double t){
        return a+(b-a)*t;
    }

    int main(){
        cout<< Physics::clamp(120.0, 0.0, 100.0)<< endl;
        cout<< GameMath::clamp(120, 0, 100)<< endl;
        cout << Physics::lerp(0.0, 100.0, 0.5) << endl;
        cout << GameMath::lerp(0.0, 100.0, 0.5) << endl;
        {
            using namespace Physics;
            cout << clamp(50.0, 0.0, 100.0) << endl;
        }

        return 0;
    }
}

```

Output:



```

<terminal>
100
100
50
50
50

```

Part C — Dynamic 2D Game Map

Source code:

```
#include<iostream>

using namespace std;

int main(){

    int R,C;

    cout<<"Enter rows and columns: ";

    cin>>R>>C;

    cout << "===== GAME MAP (" << R << " x " << C << ") =====" << endl;

    int** map = new int*[R];

    for(int i=0;i<R;i++){

        map[i]=new int[C];

    }

    for(int i=0;i<R;i++){

        for(int j=0;j<C;j++){

            map[i][j]=rand()%5;

        }

    }

    for(int i=0;i<R;i++){

        for(int j=0;j<C;j++){

            cout<<map[i][j]<<" ";

        }

        cout<<endl;

    }

    cout<<"Legend: 0=Grass 1=Water 2=Mountain 3=Forest 4=Dungeon"<<endl;

    int count[5] = {0};

    for(int i=0;i<R;i++){

        for(int j=0;j<C;j++){

            count[map[i][j]]++;

        }

    }

}
```

```

        cout << "Tile Count:" << endl;

        cout << "Grass : " << count[0] << endl;

        cout << "Water : " << count[1] << endl;

        cout << "Mountain : " << count[2] << endl;

        cout << "Forest : " << count[3] << endl;

        cout << "Dungeon : " << count[4] << endl;

        for(int i=0;i<R;i++){

            delete[] map[i];

        }

        delete[] map;

        return 0;

    }

```

Output:

```

Enter rows and columns: 3
4
===== GAME MAP (3 x 4) =====
1 2 4 0
4 4 3 3
2 4 0 0
Legend: 0=Grass 1=Water 2=Mountain 3=Forest 4=Dungeon
Tile Count:
Grass : 3
Water : 1
Mountain : 2
Forest : 2
Dungeon : 4

```

Scope Resolution Bonus (Optional)

Source code:

```
#include <iostream>

using namespace std;

int level = 1;

namespace Engine{
    namespace Audio{
        void playSound(string name){
            cout<<"Playing: "<<name<<endl;
        }
    }
}

int main(){
    int level=10;

    cout<<"Player Level: "<<level<<endl;
    cout<<"Game Difficulty Level: "<<::level<<endl;
    Engine::Audio::playSound("sword_clash");

    return 0;
}
```

Output:

```
Player Level: 10
Game Difficulty Level: 1
Playing: sword_clash
```